DROP VAULT -DROPBOX REPLICA

Abdulraheem Kehinde Adedeji -3090430

GRIFFITH COLLEGE DUBLIN 23/04/2025

Table of Contents

Brief

DropVault is a simplified Dropbox clone that uses Firebase, Firestore, and Google Cloud Storage. It features intuitive navigation, modal dialogs, and collapsible sections for file management.

To start up the App follow this command on your terminal (mac)

1. source env/bin/activate
2. export GOOGLE_APPLICATION_CREDENTIALS='/Users/MRPERFECT/Desktop/Desktop /Griffith College/Year 4/CS/Assignment/DropVault/dropvault-1-service_account.json' (or change to your path where you saved my service account.)
3. uvicorn main:app --reload

Overview

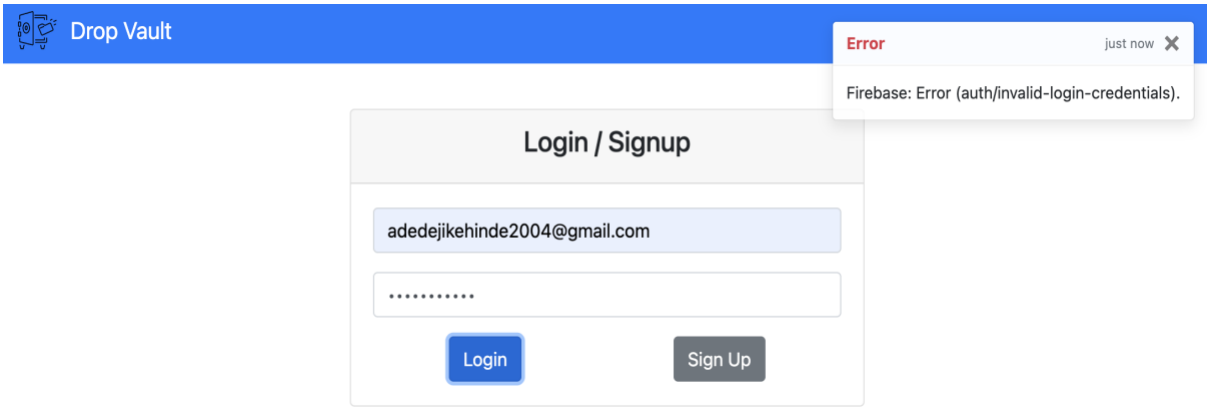
DropVault is a cloud storage solution inspired by Dropbox, built with FastAPI, Firebase, Firestore, and Google Cloud Storage. It features a user-friendly interface with Bootstrap and Jinja2, allowing secure login, folder creation, file management, and sharing.

User Interface

DropVault's user interface features a minimalist design with a two-view structure, dynamically switching between a Login View and a Main View. The Main View displays directories and files in a table layout with a fixed header.

Login UI

The Login View is designed to be simple and responsive, with two input fields for email and password. The error element is hidden by default and becomes visible with an appropriate message if login or signup fails.



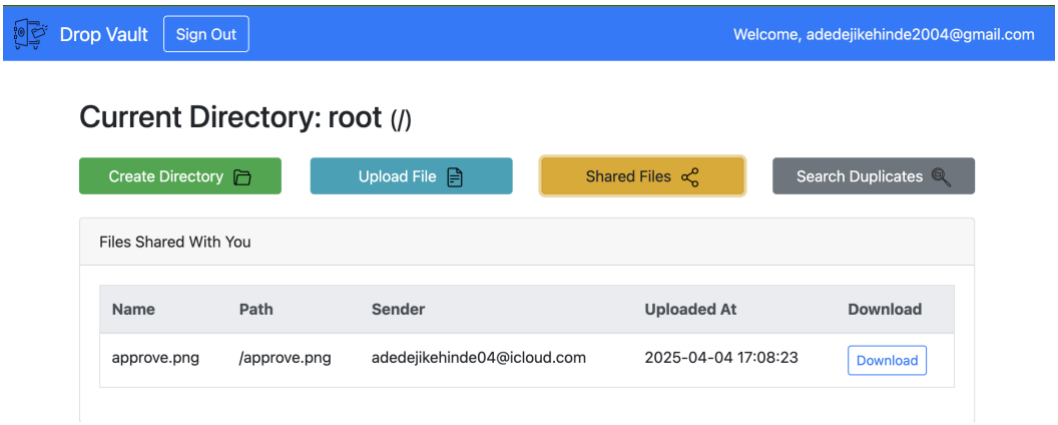
Main UI

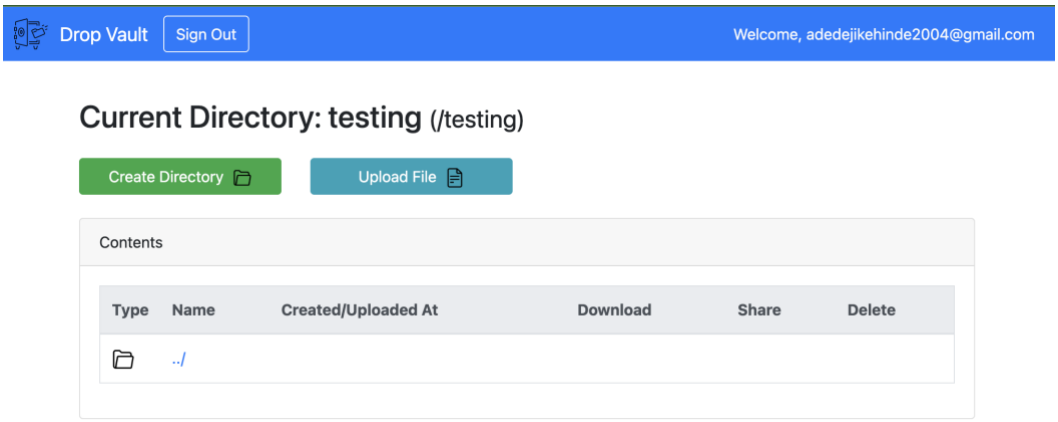
DropVault's Main View displays directories and files in a table format, with action buttons for files and navigation options for directories. The header includes a home button, user email, and sign-out option.

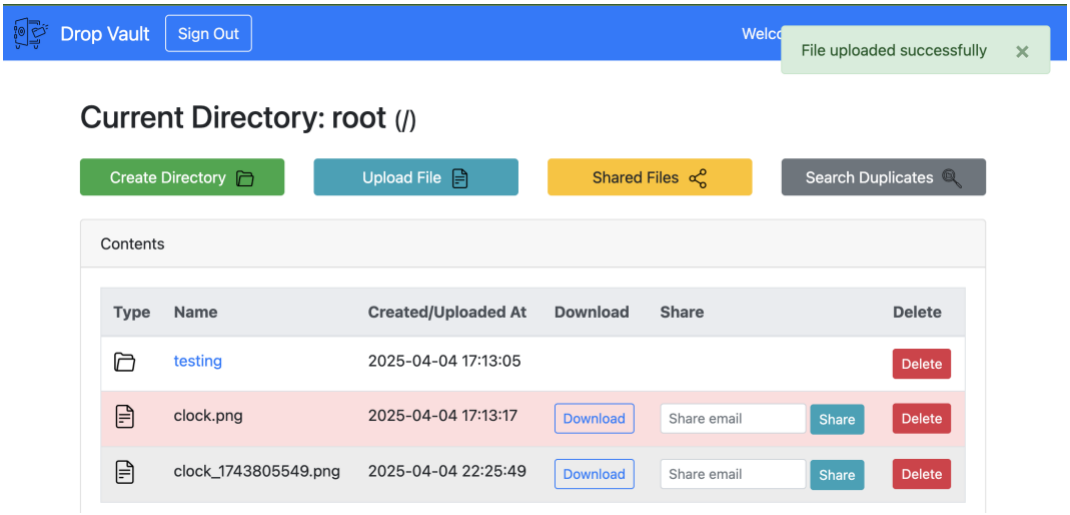
At the top, four buttons—**Create Directory**, **Upload File**, **Shared Files**, and **Search Duplicates**—allow users to perform essential tasks.

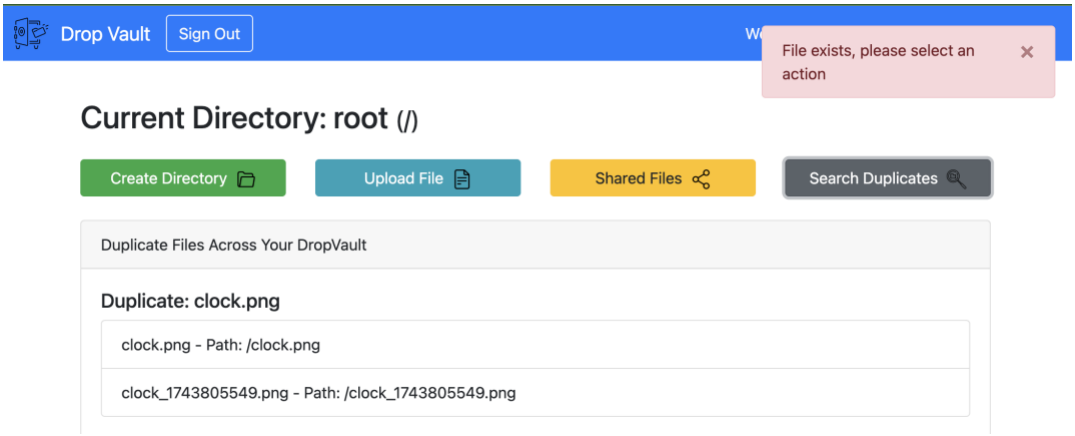
- **Create Directory** and **Upload File** open modal dialogs, keeping the main screen uncluttered while guiding the user through the creation and upload processes.
- **Shared Files** and **Search Duplicates** become especially relevant on the root page: they toggle collapsible sections that display any files shared with the user and detect duplicate files across the entire dropbox, respectively. These features remain hidden in deeper directories, ensuring the interface stays simple and focused on the current folder's contents.

Overall, the Main View combines clarity and convenience, making it straightforward for users to navigate directories, manage their files, and access advanced features like file sharing and duplicate detection.







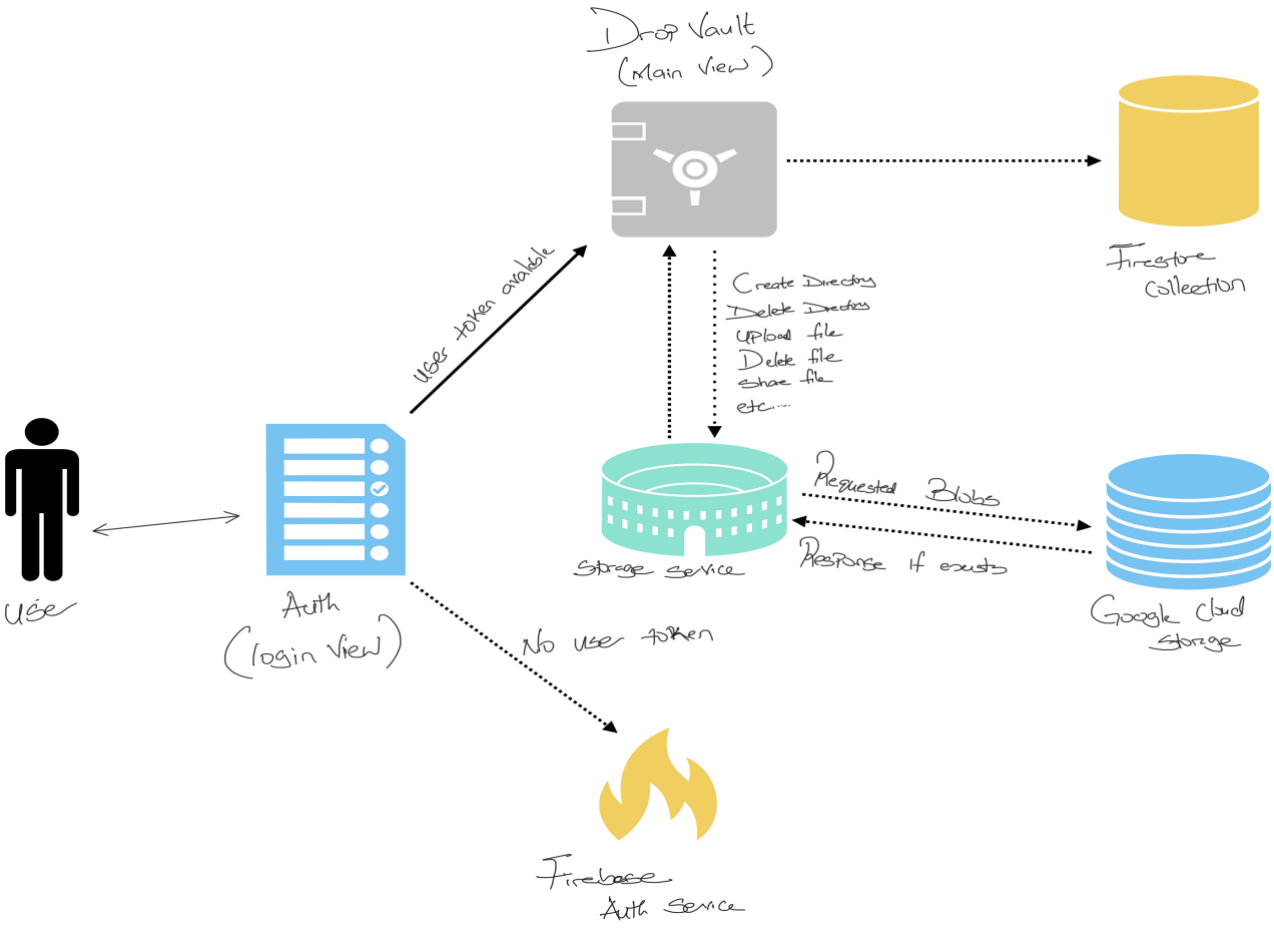


Architecture Overview

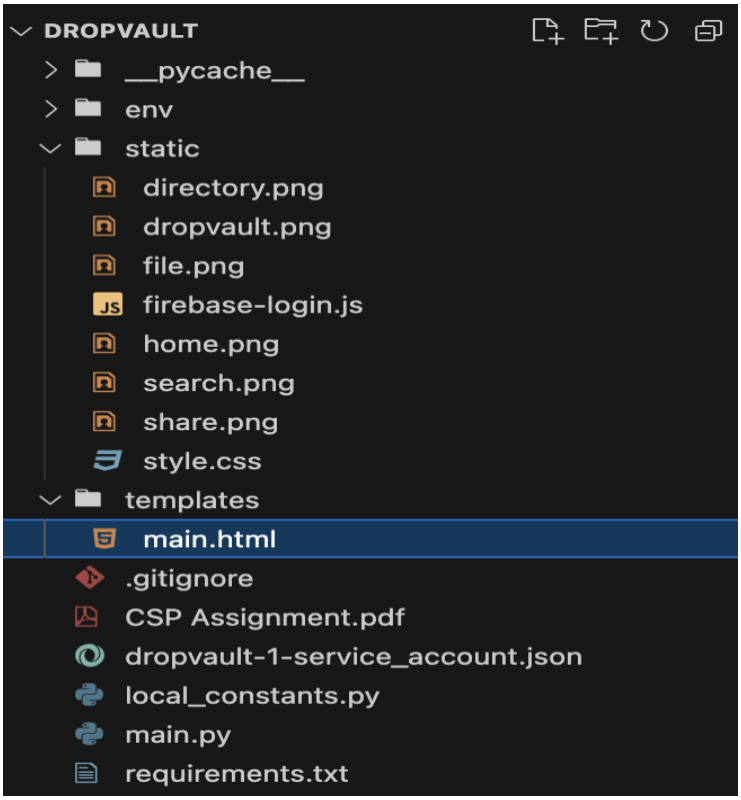
DropVault's backend is built on FastAPI, a high-performance Python framework that defines API routes rendered via Jinja2 templates. The application uses several Google Cloud services:

- **Firebase** handles user authentication by validating credentials and issuing secure tokens.
- **Firestore** stores user data and metadata for directories and files.
- **Google Cloud Storage** manages the actual file storage as blobs.

A typical request starts with user authentication via Firebase, followed by FastAPI processing actions (such as file uploads or directory creation) that update Firestore and Cloud Storage. Finally, the UI is dynamically rendered to display the updated file and directory structure.



Directory Structure



The DropVault project is organized into several folders and files, each serving a specific purpose. Below is an overview of the main directories and key files:

Static

The static folder contains unchanging files: images for the user interface, firebase-login.js for Firebase authentication, and style.css for custom styling.

Templates

This folder contains my HTML templates, which are rendered using Jinja2. The main template, **main.html**, defines the layout of the user interface for logged-in users, including tables for directories and files, modals for actions, and collapsible sections for shared and duplicate files.

Dropvault Service account file

This JSON file contains the service account credentials for Google Cloud Platform. My application uses these credentials to connect with Firestore, Cloud Storage, and other Google services.

Local Constant file

In this file, I store configuration constants like the bucket name and other settings needed throughout the project. It helps me keep all the important settings in one place.

Main.py file

This is the main entry point of my application. I built my backend using FastAPI, and all the routes (or endpoints) are defined in this file. It handles everything from user login to creating directories, uploading files, sharing files, and more.

Requirement.txt file

This file lists all the Python libraries and their versions that my project depends on. It makes it easy to set up the project on a new machine by running pip install -r requirements.txt.

Methods Overview

Routes and functions are grouped by assignment groups, with a table describing each method's purpose.

Task 1: Login/Logout Service (firebase-login.js)

Function	Description
getAuth(authInstance)	Obtains the Firebase Authentication instance from the initialized app. This instance is used for all authentication operations.
createUserWithEmailAndPassword	Listens for the "Sign Up" button click. It takes the email and password provided by the user, creates a new account via Firebase, retrieves an ID token, and sets a cookie.
signInWithEmailAndPassword	Listens for the "Login" button click. It uses the provided credentials to sign in via Firebase. Upon successful login, it retrieves an ID token and stores it in a cookie.
signOut	Listens for the "Sign Out" button click. It calls Firebase's signOut function, which clears the user's authentication state and removes the token cookie.
GET / (root route) in main.py	When the user accesses the root, this route checks for a valid token, creates a user record in Firestore (if one does not exist), and sets up a default root directory. It then renders the main view.
GET /logout in main.py	Clears the authentication token from cookies and redirects the user to the login page, ensuring a proper logout.

Task 2: Create Collections for User, Directory, and File

Function/Route	Description
get_user (decoded_token)	Checks if a user document exists in Firestore using the UID extracted from the Firebase token. If the document is not found, it creates a new user record and also creates a default root directory with the path "/". This function ensures that the required collections (User, Directory, and File) are set up for the user from the very first login.

Task 3: Create Directory

Method/Route	Description
POST /create-directory	This route handles the creation of a new directory. It receives the directory name and parent path from the user. The route first checks Firestore to ensure that no directory with the same name exists under the given parent directory. If a duplicate is found, it redirects back with an error message; otherwise, it creates a new directory document with details like user ID, name, full path, parent path, and creation timestamp, and then redirects with a success message.

Task 4 & 11: Delete Directory

Method/Route	Description
POST /delete-directory	This route handles the deletion of a directory. It first verifies that the directory belongs to the current user and checks that the directory is empty (i.e., it has no subdirectories or files). (Task 4)If the directory is not empty, it redirects back with an error message. Otherwise, it deletes the directory from Firestore and confirms success via a message.(Task 11)

Task 5, 6 & 7: Change into a Directory and Go Up a Directory

Method/Route	Description
GET /directory/{dir_id}	This route allows the user to navigate into a selected subdirectory (Task 5). When the current directory is not the root, it automatically adds a special "../" entry at the top of the table (Task 6) that links to the parent directory, allowing the user to move up one level. If the user is already at the root directory, the "../" entry is hidden (Task 7). The route fetches all the necessary directory details, subdirectories, files, and parent directory data from Firestore, then renders the main view with the updated content.

Task 8: Upload File

Method/Route	Description
POST /upload-file	This route handles the file upload process. It accepts a file and the current directory information, then checks if a file with the same name already exists in that directory. If a duplicate is found, it requires the user to specify whether to overwrite the existing file or to upload it as a duplicate (by appending a unique suffix to the filename). Once the decision is made, the file is uploaded to Google Cloud Storage, its metadata (including a computed hash, upload timestamp, and sender's email) is stored in Firestore, and the user is redirected with an appropriate success or error message.

Task 9: Delete File

Method/Route	Description
POST /delete-file	This route allows me to delete a file from the current directory. It verifies that the file belongs to the logged-in user, then deletes the file from both Google Cloud Storage and Firestore. Once the deletion is completed, the user is redirected back to the directory view with a success message. If deletion fails, an error message is shown.

Task 10: Download File

Method/Route	Description
GET /download-file/{file_id}	This route handles the file download process. It verifies that the user is either the owner of the file or is authorized via file sharing, and then retrieves the file from Google Cloud Storage. Once retrieved, the file is sent back to the client as a downloadable response with the appropriate file name and content type.

Task 12 & 13:Detect Duplicate File

Method/Route/Function	Description
mark_duplicate_files(files)	Iterates through a list of files and counts how many times each file's hash appears. If a file's hash appears more than once, it marks that file as a duplicate. This function is used both to check for duplicates in the current directory (Task 12) and across all files in the dropbox (Task 13).
group_duplicate_files(files)	Takes the list of files flagged as duplicates and groups them by their hash values. It also extracts a base filename (by removing any appended duplicate suffix) to serve as the title for each duplicate group. For Task 13, this function is applied to the entire set of files across the user's dropbox, allowing for a global duplicate detection.

Additional Helper Functions

Function	Description
verify_firebase_token (id_token)	Validates the Firebase ID token using Google's Authentication library and returns the decoded token if valid. This ensures every request is authenticated.
query_files(user_id, directory_path)	Retrieves all file metadata for a specific directory from Firestore. It's used to display files in the current directory, supporting file operations like download and delete.
query_all_files (user_id)	Retrieves all files for a given user across all directories from Firestore. This is primarily used for global operations such as duplicate file detection.
query_shared_files (user_email)	Fetches files from Firestore that have been shared with the given user by checking the shared_with field. This helps populate the Shared Files section in the UI.

Bibliography

FastAPI, FastAPI Documentation, available at: https://fastapi.tiangolo.com/

Google, Google App Engine Documentation, available at: https://cloud.google.com/appengine/docs

Bootstrap, n.d., Bootstrap Documentation, available at: https://getbootstrap.com/docs/4.5/getting-started/introduction/

FlatIcon, Free Icons, available at: https://www.flaticon.com/

Griffith College Dublin, n.d., Moodle Notes and Labs for Cloud Services & Platforms